

Домашнее задание к занятию «Введение в Terraform»

1. Скачайте и установите **Terraform** версии $\geq 1.12.0$. Приложите скриншот вывода команды `terraform --version` .
2. Скачайте на свой ПК этот git-репозиторий. Исходный код для выполнения задания расположен в директории **01/src**.
3. Убедитесь, что в вашей ОС установлен docker.

```
• vboxuser@KVA:/tmp$ terraform version
Terraform v1.15.3
on linux_amd64
• vboxuser@KVA:/tmp$ docker --version
Docker version 29.4.2, build 055a478
○ vboxuser@KVA:/tmp$
```

Задание 1

1. Перейдите в каталог [src](#). Скачайте все необходимые зависимости, использованные в проекте.
2. Изучите файл **.gitignore**. В каком terraform-файле, согласно этому .gitignore, допустимо сохранить личную, секретную информацию?(логины,пароли,ключи,токены итд)
3. Выполните код проекта. Найдите в state-файле секретное содержимое созданного ресурса **random_password**, пришлите в качестве ответа конкретный ключ и его значение.

4. Раскомментируйте блок кода, примерно расположенный на строчках 29–42 файла **main.tf**.
Выполните команду `terraform validate`. Объясните, в чём заключаются намеренно допущенные ошибки. Исправьте их.
5. Выполните код. В качестве ответа приложите: исправленный фрагмент кода и вывод команды `docker ps`.
6. Замените имя docker-контейнера в блоке кода на `hello_world`. Не перепутайте имя контейнера и имя образа. Мы всё ещё продолжаем использовать `name = "nginx:latest"`.
Выполните команду `terraform apply -auto-approve`. Объясните своими словами, в чём может быть опасность применения ключа `-auto-approve`. Догадайтесь или нагуглите зачем может пригодиться данный ключ? В качестве ответа дополнительно приложите вывод команды `docker ps`.
7. Уничтожьте созданные ресурсы с помощью **terraform**. Убедитесь, что все ресурсы удалены. Приложите содержимое файла **terraform.tfstate**.
8. Объясните, почему при этом не был удалён docker-образ **nginx:latest**.
Ответ **ОБЯЗАТЕЛЬНО НАЙДИТЕ В ПРЕДОСТАВЛЕННОМ КОДЕ**, а затем **ОБЯЗАТЕЛЬНО ПОДКРЕПИТЕ** строчкой из документации [terraform провайдера docker](#). (ищите в классификаторе resource docker_image)

Разбор задания:

1. Скачал все необходимые зависимости, использованные в проекте

2. Личная информация (секреты, пароли, ключи и др.) хранятся в папке `personal.auto.tfvars` : auto - подхватывается terraform без явного указания, самостоятельно, -vars - variables, переменные. Папка создается руками (в отличие от `tfstate` , создаваемой автоматически, куда пишется состояние terraform, где тоже могут содержаться пароли ресурсов, но не вносимые руками).

3. Выполнил код

```
• vboxuser@KVA:~/terraform/src$ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of kreuzwerker/docker...
- Finding latest version of hashicorp/random...
- Installing kreuzwerker/docker v4.3.0...
- Installed kreuzwerker/docker v4.3.0 (unauthenticated)
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (unauthenticated)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Warning: Incomplete lock file information for providers

Due to your customized provider installation methods, Terraform was forced to calculate lock file checksums locally for the following
providers:
- hashicorp/random
- kreuzwerker/docker

The current .terraform.lock.hcl file only includes checksums for linux_amd64, so Terraform running on another platform will fail to install
these providers.

To calculate additional checksums for another platform, run:
  terraform providers lock -platform=linux_amd64
(where linux_amd64 is the platform to generate)

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
• vboxuser@KVA:~/terraform/src$
```

```
• vboxuser@KVA:~/terraform/src$ terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# random_password.random_string will be created
+ resource "random_password" "random_string" {
  + bcrypt_hash = (sensitive value)
  + id          = (known after apply)
  + length      = 16
  + lower       = true
  + min_lower   = 1
  + min_numeric = 1
  + min_special = 0
  + min_upper   = 1
  + number      = true
  + numeric     = true
  + result      = (sensitive value)
  + special     = false
  + upper       = true
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

random_password.random_string: Creating...

random_password.random_string: Creation complete after 0s [id=none]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```
• vboxuser@KVA:~/terraform/src$
```

пароль тут:

```
5 "lineage": "0fa454c0-dfe2-a159-2357-46682aea1747",
6 "outputs": {},
7 "resources": [
8   {
9     "mode": "managed",
10    "type": "random_password",
11    "name": "random_string",
12    "provider": "provider[\"registry.terraform.io/hashicorp/random\"]",
13    "instances": [
14      {
15        "schema_version": 3,
16        "attributes": {
17          "bcrypt_hash": "$2a$10$eWDilmFrQqd10IvHWyDIWuSfQU0TFEeMMfHFz0U9anXIAWtNTRup6",
18          "id": "none",
19          "keepers": null,
20          "length": 16,
21          "lower": true,
22          "min_lower": 1,
23          "min_numeric": 1,
24          "min_special": 0,
25          "min_upper": 1,
26          "number": true,
27          "numeric": true,
28          "override_special": null,
29          "result": "dupw6E7KbrcaljDz",
30          "special": false,
31          "upper": true
32        },
33        "sensitive_attributes": [
34          [
35            {
36              "type": "get_attr",
37              "value": "bcrypt_hash"
38            }
39          ]
40        ]
41      }
42    ]
43  }
44 ]
```

🔖 Bookmark

"result": "dupw6E7KbrcaljDz"

4. Раскоментировал.

```
vboxuser@KVA:~/terraform/src$ terraform validate

Error: Missing name for resource

  on main.tf line 23, in resource "docker_image":
 23: resource "docker_image" {

All resource blocks must have 2 labels (type, name).

Error: Invalid resource name

  on main.tf line 28, in resource "docker_container" "nginx":
 28: resource "docker_container" "nginx" {

A name must start with a letter or underscore and may contain only letters, digits, underscores, and dashes.

vboxuser@KVA:~/terraform/src$
```

5. Исправил

```
1
2 resource "docker_image" "nginx" {
3     name          = "nginx:latest"
4     keep_locally = true
5 }
6
7 resource "docker_container" "nginx" {
```

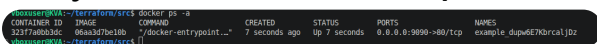
```

8  image = docker_image.nginx.image_id
9  name   = "example_${random_password.random_string.result}"
10
11  ports {
12      internal = 80
13      external = 9090
14  }
15 }

```

Ошибки следующие:

- не было имени докер образа nginx. Добавил "nginx"
- в названии ресурса "1nginx" перед nginx стояла цифра 1, убрал
- в строке `name = "example_${random_password.random_string_FAKE.result}"` значения посталил как в tfstate файле: `random_password.random_string`, без FAKE, значение `result` было написано с ошибкой (большая буква T, сменил на маленькую). Код запустился, контейнер поднялся.



```

houshser@vps:~/terraform/src$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
2237f80b36c   0ba3d76210b   7/docker-entrypoint...   7 seconds ago   Up 7 seconds   0.0.0.0:9090->80/tcp       example-dupwE7brcalJ2z
houshser@vps:~/terraform/src$

```

6. Заменял имя контейнера на *helloworld*. Почему тут:

resource "docker_image" "nginx" - имя ресурса внутри terraform,

name = "nginx:latest" - имя докер образа в Docker hub

resource "docker_container" "nginx" - имя ресурса в terraform

image = docker_image.nginx.image_id - ссылка на ресурс, создаваемый terraform

`name = "hello_world${random_password.random_string.result}"` - имя докер - контейнера в докер

```
26 }
27
28 resource "docker_container" "nginx" {
29   image = docker_image.nginx.image_id
30   name = "hello_world${random_password.random_string.result}"
31
32   ports {
33     internal = 80
34     external = 9090
35   }
36 }
```

выполнил команду `terraform apply -auto-approve`

```
vboxuser@KVA:~/terraform/src$ terraform apply -auto-approve
# (1 unchanged block hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.
docker_container.nginx: Destroying... [id=e00da2ad3942d5c341e9a37a4e2e0c7ac0206220abb681e5c0036a499d836cfa]
docker_container.nginx: Destruction complete after 0s
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 1s [id=4f8dcd2a18399636cd8c57c77a80ece9eef10de24ff8368dd6f6944eae4bdb37]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
vboxuser@KVA:~/terraform/src$
```

Terraform примени изменения без дополнительной верификации и ввода команды подтверждения `yes`

Если внести изменения и скомандовать `terraform apply` то будет выведен весь план изменений и команда подтверждения:

```

vboxuser@KVA:~/terraform/src$ terraform apply
    - network_name           = "bridge"
      # (2 unchanged attributes hidden)
  },
] -> (known after apply)
~ platform                  = "linux" -> (known after apply)
- privileged                = false -> null
- publish_all_ports         = false -> null
~ runtime                   = "runc" -> (known after apply)
~ security_opts              = [] -> (known after apply)
~ shm_size                   = 64 -> (known after apply)
~ stop_signal                = "SIGQUIT" -> (known after apply)
~ stop_timeout               = 0 -> (known after apply)
- storage_opts               = {} -> null
- sysctls                    = {} -> null
- tmpfs                     = {} -> null
  # (22 unchanged attributes hidden)

~ healthcheck (known after apply)

~ labels (known after apply)

  # (1 unchanged block hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

docker_container.nginx: Destroying... [id=4f8dcd2a18399636cd8c57c77a80ece9eef10de24ff8368dd6f6944eae4bdb37]
docker_container.nginx: Destruction complete after 0s
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 0s [id=3eb91e6ca9c89ff5e22885f1687485b6cd59396373905d643393c9af89fdc66a]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
vboxuser@KVA:~/terraform/src$

```

Этот сценарий с auto-approve применим тогда, когда вводить руками yes некому. Например при

применении скриптов автоматизации, в CI/CD конвейере и просто при ускорении выполнения команды apply.

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
• vboxuser@KVA:~/terraform/src$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
a21399225df7   06aa3d7be10b   "/docker-entrypoint..." 10 seconds ago Up 9 seconds   0.0.0.0:9090->80/tcp     hello_world_dupw6E7KbrcaljDz
○ vboxuser@KVA:~/terraform/src$
```

6. Уничтожил ресурсы командой `terraform destroy` :

```

vboxuser@KVA:~/terraform/src$ terraform destroy
- keep_locally = true -> null
- name         = "nginx:latest" -> null
- repo_digest  = "nginx@sha256:06aa3d7be10bc6307990c81bdca075793132e9163391abc370c015e344e23128" -> null
}

# random_password.random_string will be destroyed
- resource "random_password" "random_string" {
  - bcrypt_hash = (sensitive value) -> null
  - id         = "none" -> null
  - length     = 16 -> null
  - lower      = true -> null
  - min_lower  = 1 -> null
  - min_numeric = 1 -> null
  - min_special = 0 -> null
  - min_upper  = 1 -> null
  - number     = true -> null
  - numeric    = true -> null
  - result     = (sensitive value) -> null
  - special    = false -> null
  - upper      = true -> null
}

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

docker_container.nginx: Destroying... [id=a21399225df743e280573d235d516bbc433038c17aac1913efe60c4c42d49c8d]
docker_container.nginx: Destruction complete after 1s
docker_image.nginx: Destroying... [id=sha256:06aa3d7be10bc6307990c81bdca075793132e9163391abc370c015e344e23128nginx:latest]
random_password.random_string: Destroying... [id=none]
docker_image.nginx: Destruction complete after 0s
random_password.random_string: Destruction complete after 0s

Destroy complete! Resources: 3 destroyed.
vboxuser@KVA:~/terraform/src$

```

содержимое файла `terraform.tfstate`:

```
main.tf  terraform.tfstate X
terraform.tfstate
1  {
2    "version": 4,
3    "terraform_version": "1.15.3",
4    "serial": 23,
5    "lineage": "0fa454c0-dfe2-a159-2357-46682aea1747",
6    "outputs": {},
7    "resources": [],
8    "check_results": null
9  }
10
```

```
vboxuser@KVA:~/terraform/src$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	06aa3d7be10b	240MB	65.9MB	

образ на месте, поскольку в коде файла main.tf явно указан булевой флаг `keep_locally = true`, который указывает на то, что образ удалять не надо, поскольку сценарии удаления и пересоздания инфраструктуры частые, а вот постоянное скачивание тяжелых образов для этого нецелесообразно. при необходимости удаления и образов тоже применяется булевое значение `keep_locally = false`, тогда образ будет стерт тоже.

main.tf

terraform.tfstate

main.tf

```
1 terraform {
2   required_providers {
3     docker = {
4       source = "kreuzwerker/docker"
5     }
6   }
7   required_version = "~>1.15.0" /*Многострочный комментарий.
8   Требуемая версия terraform */
9 }
10 provider "docker" {}
11
12 #однострочный комментарий
13
14 resource "random_password" "random_string" {
15   length      = 16
16   special     = false
17   min_upper   = 1
18   min_lower   = 1
19   min_numeric = 1
20 }
21
22
23 resource "docker_image" "nginx" {
24   name = "nginx:latest"
25   keep_locally = true
26 }
27
28 resource "docker_container" "nginx" {
29   image = docker_image.nginx.image_id
30   name = "hello_world_${random_password.random_string.result}"
31
32   ports {
33     internal = 80
34     external = 9090
35   }
36 }
```

```
35 }  
36 }  
37
```

Вот выдержка из документации:

<https://library.tf/providers/kreuzwerker/docker/latest/docs/resources/image>

- `force_remove` (Boolean) If true, then the image is removed forcibly when the resource is destroyed.
- `keep_locally` (Boolean) If true, then the Docker image won't be deleted on destroy operation. If this is false, it will delete the image from the docker local storage on destroy operation.
- `platform` (String) The platform to use when pulling the image. Defaults to the platform of the current machine.
- `pull_triggers` (Set of String) List of values which cause an image pull when changed. This is used to store the image digest from the registry when using the [docker_registry_image](#).
- `timeouts` (Block, Optional) (see [below for nested schema](#))
- `triggers` (Map of String) A map of arbitrary strings that, when changed, will force the `docker_image` resource to be replaced. This can be used to rebuild an image when contents of source code folders change

Задание 2*

1. Создайте в облаке VM. Сделайте это через web-консоль, чтобы не слить по незнанию токен от облака в github(это тема следующей лекции). Если хотите - попробуйте сделать это через terraform, прочитав документацию yandex cloud. Используйте файл `personal.auto.tfvars` и гитигнор или иной, безопасный способ передачи токена!
2. Подключитесь к VM по ssh и установите стек docker.
3. Найдите в документации docker provider способ настроить подключение terraform на вашей рабочей станции к remote docker context вашей VM через ssh.
4. Используя terraform и remote docker context, скачайте и запустите на вашей VM контейнер `mysql:8` на порту `127.0.0.1:3306`, передайте ENV-переменные. Сгенерируйте разные пароли через `randompassword` и передайте их в контейнер, используя интерполяцию из примера с `nginx`. (`name = "example${random_password.random_string.result}"`, двойные кавычки и фигурные скобки обязательны!)

```
1 environment:
2   - "MYSQL_ROOT_PASSWORD=${...}"
3   - MYSQL_DATABASE=wordpress
4   - MYSQL_USER=wordpress
5   - "MYSQL_PASSWORD=${...}"
6   - MYSQL_ROOT_HOST="%"
```

6. Зайдите на вашу VM , подключитесь к контейнеру и проверьте наличие секретных env-переменных с помощью команды `env` . Запишите ваш финальный код в репозиторий.

Разбор решения Задачи 2*

создаю VM в Yandex Cloud через возможности yc:

Для этого пишу bash скрипт `create_VM.sh`

```
1  echo "kva-vm"
2  yc compute instance create --name kva-vm\
3  --zone ru-central1-a\
4  --network-interface subnet-name=default-ru-central1-a,nat-ip-version=ipv4\
5  --create-boot-disk image-folder-id=standard-images,image-family=ubuntu-2204-
   lts,size=20\
6  --memory 2\
7  --cores 2\
8  --metadata-from-file user-data=cloud-init.yaml
```

и конфигурационный файл `cloud-init.yaml`

```
1  #cloud-config
2  users:
3    - name: kva-vm
4    groups: sudo, docker
```

```
5     shell: /bin/bash
6     sudo: ['ALL=(ALL) NOPASSWD:ALL']
7     ssh_authorized_keys:
8         - ssh-ed25519
9         AAAAC3NzaC1lZDI1NTE5AAAAICGCxz6Ttx4jaiMvpdANwl4va+6RTIWd+Vd8yLDNu1xD  vboxuser@KVA
10
11     package_update: true
12     package_upgrade: true
13
14
15     packages:
16         - ca-certificates
17         - curl
18         - gnupg
19         - lsb-release
20
21
22     runcmd:
23         - curl -fsSL https://get.docker.com -o /tmp/get-docker.sh
24         - sh /tmp/get-docker.sh
25         - usermod -aG docker kva-vm
26         - systemctl enable docker
27         - systemctl start docker
```

```
- rm /tmp/get-docker.sh
```

```
vboxuser@KVA:~/terraform/mysql$ ./create VM.sh
```

```
o vboxuser@KVA:~/terraform/mysql$ ssh -l kva-vm 51.250.95.167
The authenticity of host '51.250.95.167 (51.250.95.167)' can't be established.
ED25519 key fingerprint is SHA256:2mMPyUd7TML/FxfYuUsek1a0zmEBB4rRjie7uLpFVzY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '51.250.95.167' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-177-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro
```

System information as of Sun May 17 11:49:26 UTC 2026

System load:	0.22	Processes:	102
Usage of /:	12.1% of 18.73GB	Users logged in:	0
Memory usage:	12%	IPv4 address for eth0:	10.128.0.22
Swap usage:	0%		

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "`sudo <command>`".
See "`man sudo_root`" for details.

после этого подключаюсь к консоли облачной VM через ssh и проверяю наличие установленного docker. Это также позволит не прописывать в файле main.tf в блоке `provider` строку `ssh_opts = ["-o", "StrictHostKeyChecking=no", "-o", "UserKnownHostsFile=/dev/null"]`, которая отключает запрос подтверждения при первом подключении к хосту. В этом случае ключ сохранится в `known_hosts` и больше запросов подтверждения не будет.

```
kva-vm@fhmofrk7nsk15h95k9r6:~$ docker version
Client: Docker Engine - Community
Version:      29.5.0
API version:  1.54
Go version:   go1.26.3
Git commit:   98f1464
Built:        Thu May 14 14:40:28 2026
OS/Arch:      linux/amd64
Context:      default

Server: Docker Engine - Community
Engine:
Version:      29.5.0
API version:  1.54 (minimum version 1.40)
Go version:   go1.26.3
Git commit:   ff8d90a
Built:        Thu May 14 14:40:28 2026
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      v2.2.3
GitCommit:    77c84241c7cbdd9b4eca2591793e3d4f4317c590
runc:
Version:      1.3.5
GitCommit:    v1.3.5-0-g488fc13e
docker-init:
Version:      0.19.0
GitCommit:    de40ad0
kva-vm@fhmofrk7nsk15h95k9r6:~$
```

создаю подпапку: `/home/vboxuser/terraform/mysql`, в ней будут файлы: (помимо вышеописанных скрипта и yaml конфига) `.gitignore`, `main.tf`, `personal.auto.tfvars`

```
1 # содержимое файла .gitignore
2 .terraform/
3 .terraform.lock.hcl
4 terraform.tfstate
5 terraform.tfstate.backup
6 personal.auto.tfvars
```

```
1 # содержимое файла personal.auto.tfvars
2 docker_host = "ssh://kva-vm@51.250.95.167:22"
```

```
1 # содержимое файла main.tf
2 terraform {
3     required_version = "~>1.15.0"
4
5     required_providers {
6         docker = {
7             source = "kreuzwerker/docker"
8         }
9     }
10 }
```

```
11
12 provider "docker" {
13     host = var.docker_host
14
15 }
16
17 variable "docker_host" {
18     type          = string
19     description = "Remote Docker host via SSH. Example: ssh://kva-
vm@51.250.10.10:22" #
20 }
21
22
23 resource "random_password" "mysql_root_password" {
24     length      = 16
25     special     = false
26 }
27
28 resource "random_password" "mysql_user_password" {
29     length      = 16
30     special     = false
31 }
32
33
```

```
34 resource "docker_image" "mysql" {
35     name          = "mysql:8"
36     keep_locally = false #
37 }
38
39 resource "docker_container" "mysql" {
40     name = "exemple_${random_password.mysql_user_password.result}"
41     image = docker_image.mysql.image_id
42
43     ports {
44         internal = 3306
45         external = 3306
46         ip       = "127.0.0.1"
47     }
48
49     env = [
50         "MYSQL_ROOT_PASSWORD=${random_password.mysql_root_password.result}",
51         "MYSQL_DATABASE=wordpress",
52         "MYSQL_USER=wordpress",
53         "MYSQL_PASSWORD=${random_password.mysql_user_password.result}",
54         "MYSQL_ROOT_HOST=%"
55     ]
56
57     restart = "unless-stopped"
```

```
58 }
59
60
61 output "container_name" {
62     value = docker_container.mysql.name
63 }
64
65 output "mysql_root_password" {
66     value      = random_password.mysql_root_password.result
67     sensitive = true
68 }
69
70 output "mysql_user_password" {
71     value      = random_password.mysql_user_password.result
72     sensitive = true
73 }
```

В консоли хоста: `terraform init`, `terraform validate`, `terraform plan`

```
• vboxuser@KVA:~/terraform/mysql$ terraform plan
```

Terraform used the selected providers to generate the following execution plan
+ create

Terraform will perform the following actions:

docker_container.mysql will be created

```
+ resource "docker_container" "mysql" {  
  + attach                = false  
  + bridge                = (known after apply)  
  + command               = (known after apply)  
  + container_logs        = (known after apply)  
  + container_read_refresh_timeout_milliseconds = 15000  
  + entrypoint            = (known after apply)  
  + env                   = (sensitive value)  
  + exit_code             = (known after apply)  
  + hostname              = (known after apply)  
  + id                    = (known after apply)  
  + image                 = (known after apply)  
  + init                  = (known after apply)  
  + ipc_mode              = (known after apply)  
  + log_driver            = (known after apply)  
  + logs                  = false  
  + memory_reservation    = 0  
  + must_run              = true  
  + name                  = (sensitive value)  
  + network_data          = (known after apply)  
  + network_mode          = "bridge"  
  + platform              = (known after apply)  
  + read_only             = false  
  + remove_volumes        = true  
  + restart               = "unless-stopped"  
  + rm                    = false  
  + runtime               = (known after apply)  
  + security_opts         = (known after apply)  
  + shm_size              = (known after apply)  
  + start                 = true  
  + stdin_open            = false  
  + stop_signal           = (known after apply)  
  + stop_timeout          = (known after apply)  
  + tty                   = false
```

```
+ wait          = false
+ wait timeout  = 60
```

```
+ healthcheck (known after apply)

+ labels (known after apply)

+ ports {
  + external = 3306
  + internal = 3306
  + ip       = "127.0.0.1"
  + protocol = "tcp"
}
}

# docker_image.mysql will be created
+ resource "docker_image" "mysql" {
  + id          = (known after apply)
  + image_id    = (known after apply)
  + keep_locally = false
  + name        = "mysql:8"
  + repo_digest = (known after apply)
}

# random_password.mysql_root_password will be created
+ resource "random_password" "mysql_root_password" {
  + bcrypt_hash = (sensitive value)
  + id          = (known after apply)
  + length      = 16
  + lower       = true
  + min_lower   = 0
  + min_numeric = 0
  + min_special = 0
  + min_upper   = 0
  + number      = true
  + numeric     = true
  + result      = (sensitive value)
  + special     = false
  + upper       = true
}

# random_password.mysql_user_password will be created
+ resource "random_password" "mysql_user_password" {
  + bcrypt_hash = (sensitive value)
```

```
+ id      = (known after apply)
+ length  = 16
+ lower   = true
+ min_lower = 0
+ min_numeric = 0
+ min_special = 0
+ min_upper = 0
+ number  = true
+ numeric = true
+ result  = (sensitive value)
+ special = false
+ upper   = true
}
```

Plan: 4 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ container_name      = (sensitive value)
+ mysql_root_password = (sensitive value)
+ mysql_user_password = (sensitive value)
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

В консоли хоста: `terraform apply` :

Enter a value: yes

```
docker_image.mysql: Creating...
docker_image.mysql: Still creating... [00m10s elapsed]
docker_image.mysql: Still creating... [00m20s elapsed]
docker_image.mysql: Creation complete after 26s [id=sha256:c36050afdca850f23cef85703f84c7531a5ae155a11b5ee1c60acb09937c4084mysql:8]
docker_container.mysql: Creating...
docker_container.mysql: Creation complete after 2s [id=106512e58f17c5c3950ef6dcf3d229af896da9859b9c1d4f9ea1e3e0c1a6a090]
```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```
container_name = <sensitive>
mysql_root_password = <sensitive>
mysql_user_password = <sensitive>
```

vboxuser@KVA:~/terraform/mysql\$

Проверка на ВМ

```
1 ssh kva-vm@51.250.95.167
2 docker ps
3 docker exec -it exemple_4MzHKBacnHYEMvbL env
```

```
kva-vm@fhmofrk7nsl5h95k9r6:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
106512e58f17   c36050afdca8                       "docker-entrypoint.s..." About a minute ago Up About a minute 127.0.0.1:3306->3306/tcp, 33060/tcp  exemple_4MzHKBacnHYEMvbL
kva-vm@fhmofrk7nsl5h95k9r6:~$
kva-vm@fhmofrk7nsl5h95k9r6:~$ docker exec -it exemple_4MzHKBacnHYEMvbL env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=106512e58f17
TERM=xterm
MYSQL_ROOT_HOST=%
MYSQL_ROOT_PASSWORD=v9qwTvIb2yM1cG6it
MYSQL_PASSWORD=4MzHKBacnHYEMvbL
MYSQL_DATABASE=wordpress
MYSQL_USER=wordpress
GOSU_VERSION=1.19
MYSQL_MAJOR=8.4
MYSQL_VERSION=8.4.9-1.el9
MYSQL_SHELL_VERSION=8.4.9-1.el9
HOME=/root
kva-vm@fhmofrk7nsl5h95k9r6:~$
```

Задача 3*

1. Установите [opentofu](#) (fork terraform с лицензией Mozilla Public License, version 2.0) любой версии
2. Попробуйте выполнить тот же код с помощью `tofu apply`, а не `terraform apply`

Разбор Задачи 3*

Установил **opentofu** по инструкции из официального ресурса:

<https://opentofu.org/docs/intro/install/deb/>

```
• vboxuser@KVA:~$ which tofu
  /usr/bin/tofu
• vboxuser@KVA:~$ tofu version
  OpenTofu v1.12.0
  on linux_amd64
○ vboxuser@KVA:~$
```

При помощи специального ПО для подмены региона нахождения пользователя, скачал и Tofu и его Registry

```
vboxuser@KVA:~/terraform/mysql$ tofu init
- Finding latest version of kreuzwerker/docker...
- Reusing previous version of hashicorp/random from the dependency lock file
- Installing hashicorp/random v3.9.0...
- Installing kreuzwerker/docker v4.4.0...
- Installed hashicorp/random v3.9.0 (signed, key ID 0C0AF313E5FD9F80)
- Installed kreuzwerker/docker v4.4.0 (signed, key ID 0DCE698927DAF8EC)
```

Providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
<https://opentofu.org/docs/cli/plugins/signing/>

OpenTofu has made some changes to the provider dependency selections recorded in the .terraform.lock.hcl file. Review those changes and commit them to your version control system if they represent changes you intended to make.

Warning: Dependency lock file entries automatically updated

OpenTofu automatically rewrote some entries in your dependency lock file:

- registry.terraform.io/hashicorp/random => registry.opentofu.org/hashicorp/random

The version selections were preserved, but the hashes were not because the OpenTofu project's provider releases are not byte-for-byte identical.

OpenTofu has been successfully initialized!

You may now begin working with OpenTofu. Try running "tofu plan" to see any changes that are required for your infrastructure. All OpenTofu commands should now work.

If you ever set or change modules or backend configuration for OpenTofu, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
vboxuser@KVA:~/terraform/mysql$
```

на команде `tofu plan` отсутствует соединение с ВМ в Yandex Cloud (как с ВПН так и без него)

```
vboxuser@KVA:~/terraform/mysql$ tofu plan
random_password.mysql_root_password: Refreshing state... [id=none]
random_password.mysql_user_password: Refreshing state... [id=none]
docker_image.mysql: Refreshing state... [id=sha256:c36050afdca850f23cef85703f84c7531a5ae155a11b5ee1c60acb09937c4084mysql:8]

Planning failed. OpenTofu encountered an error while generating this plan.

Error: failed to create Docker client: Error pinging Docker server, please make sure that ssh://kva-vm@51.250.95.167:22 is reachable and has a '_ping' endpoint
. Error: error during connect: Head "http://docker.example.com/_ping": command [ssh -l kva-vm -p 22 -o ConnectTimeout=30 -T -- 51.250.95.167 docker system dial-st
dio] has exited with exit status 255, make sure the URL is valid, and Docker 18.09 or later is installed on the remote host: stderr=ssh: connect to host 51.250.95
.167 port 22: Connection refused

with docker_image.mysql,
on main.tf line 33, in resource "docker_image" "mysql":
33: resource "docker_image" "mysql" {
```

несмотря на то, что доступ к машине корректный

```
kva-vm@fhmofrk7nsl5h95k9r6:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
106512e58f17   c36050afdca8   "docker-entrypoint.s..." 2 hours ago    Up 16 minutes  127.0.0.1:3306->3306/tcp, 33060/tcp  exemple_4MzHKBacnHYEMvbl

kva-vm@fhmofrk7nsl5h95k9r6:~$
```

Настроить доступ к ВМ по SSH не представилось возможным... Но код на стадии `tofu init` отработал корректно, так что его выполнение на ВМ так же не вызывает сомнений.